Université Sultan Moulay Slimane
Ecole Supérieure de Technologie
- Beni Mellal -

STRUCTURES DE DONNÉES

Chapitre 3 : Piles et Files Dynamiques

Filière : DUT GI
Semestre : S3
Année Universitaire : 2026/2027

Pr. M. OUTANOUTE

Deux acquis, combinés

Chapitre 1

Le comportement : LIFO (pile), FIFO (file). Mais un tableau à taille fixe, MAX.

Chapitre 2

La souplesse : nœud par nœud, plus de MAX. Mais accès strictement séquentiel.

Ce chapitre : le comportement du Chapitre 1, représenté comme au Chapitre 2.

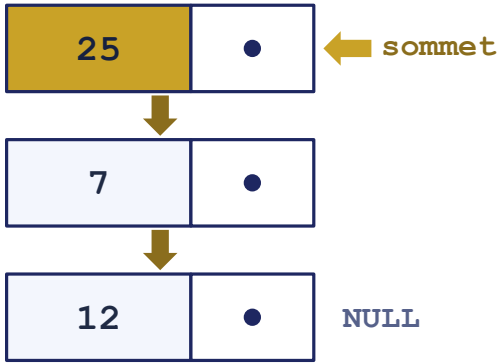
Le gain : plus de **MAX**, plus de **pilePleine**, plus de **filePleine**

— la seule limite est la mémoire réellement disponible.

La pile dynamique

```
typedef struct noeudPile {
    int valeur;
    struct noeudPile *precedent;
} NoeudPile;

NoeudPile *sommet;
```



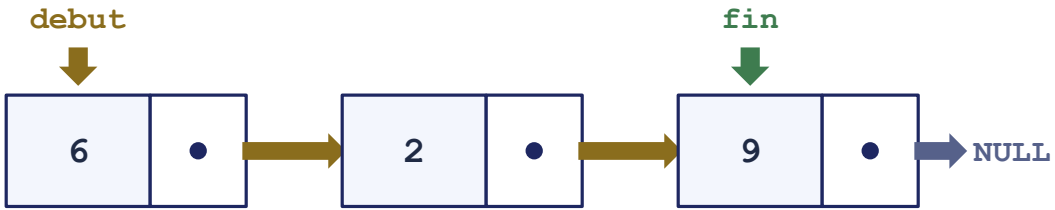
precedent *: chaque nœud pointe vers celui qui a été empilé juste avant lui.*

PARTIE B — FILE

La file dynamique

```
typedef struct noeudFile {
    int valeur;
    struct noeudFile *suivant;
} NoeudFile;

NoeudFile *debut, *fin;
```



debut *et* ***fin*** *: deux variables globales séparées —* ***fin*** *permet d'enfiler sans parcours.*

Le fil narratif : versions dynamiques

Partie A : la pile de copies — la dernière déposée est la première corrigée.

Partie B : le guichet d'inscription — le premier arrivé est le premier servi.

Les mêmes scénarios qu'au Chapitre 1, maintenant dynamiques.

Récapitulatif

- Ce chapitre combine le comportement du Chapitre 1 (***LIFO/FIFO***) et la souplesse du Chapitre 2 (***nœud par nœud***).
- Le gain : plus de ***MAX***, plus de ***pilePleine***/***filePleine***.
- Partie A — pile : nœud avec ***precedent***, repère ***sommet***.
- Partie B — file : nœud avec ***suivant***, repères ***debut*** et ***fin*** (deux variables séparées).
- Fil narratif : pile de copies (A), guichet d'inscription (B) — versions dynamiques.

PARTIE A — PILE

Déclaration

```
typedef struct noeud {
    int valeur;
    struct noeud *precedent;
} Noeud;

Noeud *sommet = NULL;
```

***sommet**, variable globale, commence à **NULL** — pile vide.*

PARTIE A — PILE

empiler

1) Créer un ***nouveau*** nœud (malloc), lui donner sa ***valeur***.
2) Faire pointer son ***precedent*** vers l'*ancien* ***sommet***.
3) ***sommet*** devient le ***nouveau*** nœud.

```
void empiler(int valeur) {
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    nouveau->precedent = sommet;
    sommet = nouveau;
}
```

Même squelette que ***insererEnTete*** du Chapitre 2 — seuls les noms changent.

PARTIE A — PILE

depiler

1) Si la pile est vide, signaler l'erreur.
2) Sinon, mémoriser le nœud ***sommet*** actuel, et *sa* ***valeur***.
3) Faire avancer ***sommet*** vers ***sommet->precedent***.
4) Libérer (free) l'*ancien nœud*, puis retourner la ***valeur*** mémorisée.

```
int depiler(void) {
    if (sommet == NULL) { printf("Pile vide\n"); return -1; }
    Noeud *ancienSommet = sommet;
    int valeur = ancienSommet->valeur;
    sommet = sommet->precedent;
    free(ancienSommet);
    return valeur;
}
```

pileVide, et détruire la pile

```
int pileVide(void) { return sommet == NULL; }
```

1) Tant que la pile n'est pas vide, *dépiler*.

```
void detruirePile(void) {
    while (sommet != NULL) depiler();
}
```

Le gain de ce chapitre : aucune fonction *pilePleine*

— une pile dynamique n'a pas de taille maximale, seule la mémoire disponible limite.

PARTIE B — FILE

Déclaration

```
typedef struct noeudFile {
    int valeur;
    struct noeudFile *suivant;
} NoeudFile;

NoeudFile *debut = NULL;
NoeudFile *fin = NULL;
```

Deux variables globales séparées, ***debut*** et ***fin*** — pas de *struct conteneur* (pour le moment).

PARTIE B — FILE

enfiler

1) Créer un ***nouveau*** *nœud*, *son* ***suivant*** à *NULL* (il sera le dernier).
2) Si la file est vide (***debut == NULL***), le ***nouveau*** *nœud* devient à la fois ***debut*** et ***fin***.
3) Sinon, faire pointer le ***suivant*** de l'*ancien* ***fin*** vers le ***nouveau*** *nœud*, puis avancer ***fin***.

```
void enfiler(int valeur) {
    NoeudFile *nouveau = malloc(sizeof(NoeudFile));
    nouveau->valeur = valeur;
    nouveau->suivant = NULL;
    if (debut == NULL) { debut = nouveau; fin = nouveau; return; }
    fin->suivant = nouveau;
    fin = nouveau;
}
```

PARTIE B — FILE

defiler

1) Si la file est vide, signaler l'erreur.
2) Sinon, mémoriser le *nœud* ***debut*** *actuel*, et *sa* ***valeur***.
3) Faire avancer ***debut*** vers ***debut->suivant***.
4) Si la file devient vide (***debut == NULL***), remettre ***fin*** à *NULL* aussi.
5) Libérer (free) l'*ancien nœud*, puis retourner la ***valeur*** mémorisée.

```
int defiler(void) {
    if (debut == NULL) { printf("File vide\n"); return -1; }
    NoeudFile *ancienDebut = debut;
    int valeur = ancienDebut->valeur;
    debut = debut->suivant;
    if (debut == NULL) { fin = NULL; }
    free(ancienDebut);
    return valeur;
}
```

PARTIE B — FILE

Le piège de cette partie

Quand ***defiler*** retire le *dernier* élément, ***debut*** devient *NULL* — mais ***fin*** continuerait de pointer vers le nœud tout juste libéré si on oublie de le réinitialiser aussi.

Les deux repères ***debut*** et ***fin*** doivent redevenir *NULL* ensemble.

Écho direct du piège de la file circulaire au Chapitre 1, où ***av*** et ***ar*** devaient déjà se réinitialiser de concert.

PARTIE B — FILE

fileVide, et détruire la file

```
int fileVide(void) { return debut == NULL; }
```

1) Tant que la file n'est pas vide, *défiler*.

```
void detruireFile(void) {
    while (debut != NULL) defiler();
}
```



Le rappel transversal : la discipline free

Rappel — Chapitre 2 :

chaque ***malloc*** engage un ***free*** correspondant.

Cette discipline s'applique identiquement aux deux structures de ce chapitre.

detruirePile et ***detruireFile*** en sont la preuve : la discipline, suivie dans les deux structures.

Récapitulatif

- Partie A (pile) : ***empiler*** = ***insererEnTete*** avec ***precedent***/***sommet*** ; ***depiler*** retourne la ***valeur***.
- Partie A : ***pileVide*** et ***detruirePile*** (réutilise ***depiler*** en boucle) ; plus de ***pilePleine***.
- Partie B (file) : ***enfiler*** en temps constant grâce à ***fin*** — plus de parcours.
- Partie B : le piège — ***defiler*** doit remettre ***fin*** à *NULL* si la file devient vide.
- Partie B : ***fileVide*** et ***detruireFile*** (réutilise ***defiler*** en boucle).
- Rappel transversal : chaque nœud supprimé, dans les deux structures, engage un ***free***.

Exercice d’application

Partie A — la pile de copies (dynamique).

1) Écrire ***empilerCopie*** et ***corrigerCopie***.
2) Dérouler l'empilement de 3 copies (101, 102, 103), puis 2 corrections.

Partie B — le guichet d'inscription (dynamique).

3) Écrire ***enregistrerArrivee*** et ***servirSuivant***.
4) Dérouler l'arrivée de 3 étudiants (201, 202, 203), puis 2 passages au guichet.

1) Le code

```
void empilerCopie(int numero) {
    empiler(numero);
}

void corrigerCopie(void) {
    if (pileVide()) { return; }
    int numero = depiler();
    printf("Correction de la copie %d\n", numero);
}
```

Des fonctions minces qui réutilisent ***empiler***/***depiler*** déjà connues — même principe qu'au Chapitre 1.

2) La trace

Étape	Action	Contenu de la pile (bas → haut)	Affichage
1	`empilerCopie(101)`	`[101]`	—
2	`empilerCopie(102)`	`[101]` ← `[102]`	—
3	`empilerCopie(103)`	`[101]` ← `[102]` ← `[103]`	—
4	**`corrigerCopie()`**	`[101]` ← `[102]`	Correction de la copie **103**
5	**`corrigerCopie()`**	`[101]`	Correction de la copie **102**

Etape 4: La copie 103, dernière empilée, est la première corrigée — LIFO, comme toujours.

3) Le code

```
void enregistrerArrivee(int numero) {
    enfiler(numero);
}

void servirSuivant(void) {
    if (fileVide()) { return; }
    int numero = defiler();
    printf("Etudiant %d servi\n", numero);
}
```

4) La trace

Étape	Action	Contenu de la file (debut → fin)	Affichage
1	`enregistrerArrivee(201)`	`[201]`	—
2	`enregistrerArrivee(202)`	`[201]` → `[202]`	—
3	`enregistrerArrivee(203)`	`[201]` → `[202]` → `[203]`	—
4	**`servirSuivant()`**	`[202]` → `[203]`	Etudiant **201** servi
5	**`servirSuivant()`**	`[203]`	Etudiant **202** servi

Etape 4: L'étudiant **201**, premier arrivé, est le premier servi — FIFO. Même nombre d'opérations que la pile, ordre inverse.

Exercice 1 :

Dans ce chapitre nous avons travaillé sur **une seule *pile*** globale.
L'exercice suivant montre où ce choix atteint sa limite — et comment la structure conteneur, écartée jusqu'ici, redevient utile dès qu'il faut plusieurs instances indépendantes.

Deux piles indépendantes — annuler et refaire :

Un éditeur de texte a besoin de **deux** piles indépendantes : une pour l'***annulation*** (*undo*), une pour le ***rétablissement*** (*redo*).

1) Proposer une structure conteneur ***Pile***, regroupant un unique repère ***sommet***.
2) Écrire une version paramétrée de ***empiler*** et de ***depiler***, opérant sur une ***Pile ** *** passée en argument.
3) Créer deux piles indépendantes, ***annulation*** et ***retablissement***, et vérifier qu'***empiler*** dans l'une n'affecte pas l'autre.

Exercice 2 :

Dans ce chapitre nous avons travaillé sur **une seule *file*** globale.
L'exercice suivant montre où ce choix atteint sa limite — et comment la structure conteneur, écartée jusqu'ici, redevient utile dès qu'il faut plusieurs instances indépendantes.

Plusieurs files indépendantes — plusieurs guichets :

L'école ouvre **deux** guichets d'inscription simultanés, chacun avec sa propre file d'attente.

1) Proposer une structure conteneur ***File***, regroupant ***debut*** et ***fin***.
2) Écrire une version paramétrée de ***enfiler*** et de ***defiler***, opérant sur une ***File ** *** passée en argument.
3) Créer deux files indépendantes, ***guichetA*** et ***guichetB***, et vérifier qu'***enfiler*** dans l'une n'affecte pas l'autre.